

Pandit Deendayal Energy University

Department of Computer Science & Engineering

Machine Learning Laboratory

Lab Assignment 1

Experiment

Data Visualization Tools Analysis

Name	Moksh Jagdish Lalwani
Roll Number	23BCP096
Division	2
Section	2
Faculty	Himanshu Gajera
Course	Machine Learning Laboratory
Department	Computer Science & Engineering

Aim

To study and compare Matplotlib, Seaborn, ggplot2 and Plotly, and understand which is best suited for a given analysis or presentation need.

Introduction

Machine learning workflows rarely begin with modelling. Before any algorithm is trained, time goes into looking at the data — checking distributions, spotting outliers, and sensing how variables relate. This lab compares four charting tools built around different philosophies: Matplotlib and Seaborn from Python, ggplot2 from R, and Plotly, used mainly when a plot needs to be interactive.

Need for Data Visualization

Raw tabular data, however clean, does not communicate patterns by itself. A dataset with a few thousand rows can hide correlations or skewed distributions that become obvious once plotted — part of why visualization is central to the exploratory data analysis (EDA) stage.

- Detecting outliers and missing-value patterns before they distort a model.
- Debugging a trained model via residual plots or confusion matrices.

The right tool depends on the audience — a static plot suffices for the developer themselves, while a stakeholder is better served by an interactive dashboard.

Comparison of Visualization Libraries

The four libraries sit at different points of the same design space. Matplotlib is the oldest and most "manual" — practically everything else in Python plotting is built on top of it. Seaborn wraps it with statistically-aware defaults; ggplot2, from R, follows the Grammar of Graphics; Plotly departs from all three by rendering charts as interactive HTML/JS objects. The next four sections cover each individually, followed by a summary table.

Matplotlib

Brief Overview

Matplotlib is the foundational plotting library for Python, first released in 2003 by John D. Hunter to give MATLAB users a similar experience. Its pyplot interface still carries that MATLAB-style syntax (`plt.plot()`, `plt.show()`).

Features

- Fine-grained control over every element of a figure — axes, ticks, legends, colours, styles.
- Supports a huge range of chart types: line, bar, scatter, histogram, contour, 3D.

Practical Use Case

Here, Matplotlib was used to plot a simple line chart tracking exam scores across student IDs, to get a feel for the basic workflow before moving to higher-level libraries.

Advantages

- Extremely flexible — almost any custom chart can eventually be made.
- Large community means most errors already have a solution online.

Limitations

- Default styling looks dated unless a style sheet is applied.
- Simple statistical plots need noticeably more code than in Seaborn.

Example Code

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(1, 11)
y = np.array([56, 62, 58, 70, 65, 72, 80, 75, 90, 85])

plt.figure(figsize=(6, 4))
plt.plot(x, y, marker='o', color='steelblue')
plt.title('Student Scores Trend')
plt.xlabel('Student ID')
plt.ylabel('Score')
plt.grid(True, alpha=0.3)
plt.show()
```

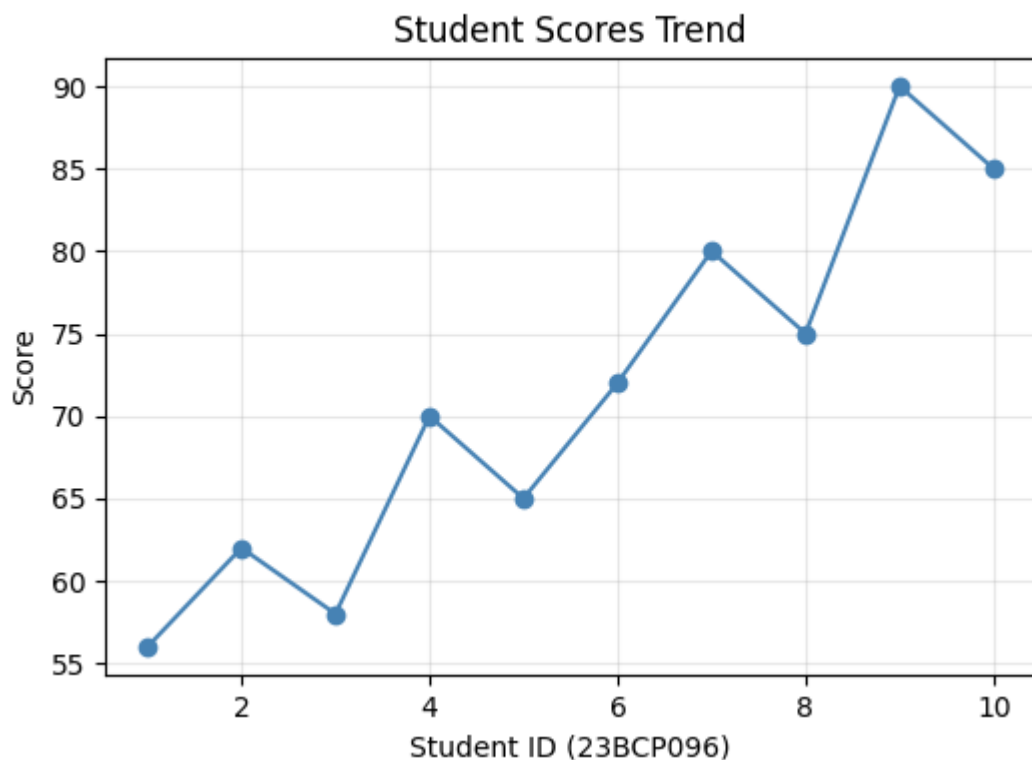


Figure 1.1: Line chart of student scores generated using Matplotlib.

Seaborn

Brief Overview

Seaborn is built on top of Matplotlib but aimed at statistical visualization. Created by Michael Waskom, it integrates tightly with pandas, so most calls take a dataframe and column names directly.

Features

- Built-in themes (darkgrid, whitegrid, ticks) that look presentable with no manual tweaking.
- High-level functions for distributions, relationships, and categorical data.

Practical Use Case

Seaborn was used to generate a correlation heatmap for a small numeric dataset — a common EDA step before feature selection.

Advantages

- Pleasant-looking charts with almost no styling effort.
- Statistical estimation, e.g. confidence intervals, is built in.

Limitations

- Deep customisation still requires dropping down to Matplotlib calls.
- Fewer chart types overall — it deliberately keeps its scope narrow.

Example Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris(as_frame=True)
df = iris.frame

# Plot
plt.figure(figsize=(8,6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="RdYlBu",
    linewidths=0.7,
    fmt=".2f",
    square=True
)
```

```
plt.title("Correlation Heatmap - Iris Dataset", fontsize=15)
plt.tight_layout()
plt.show()
```

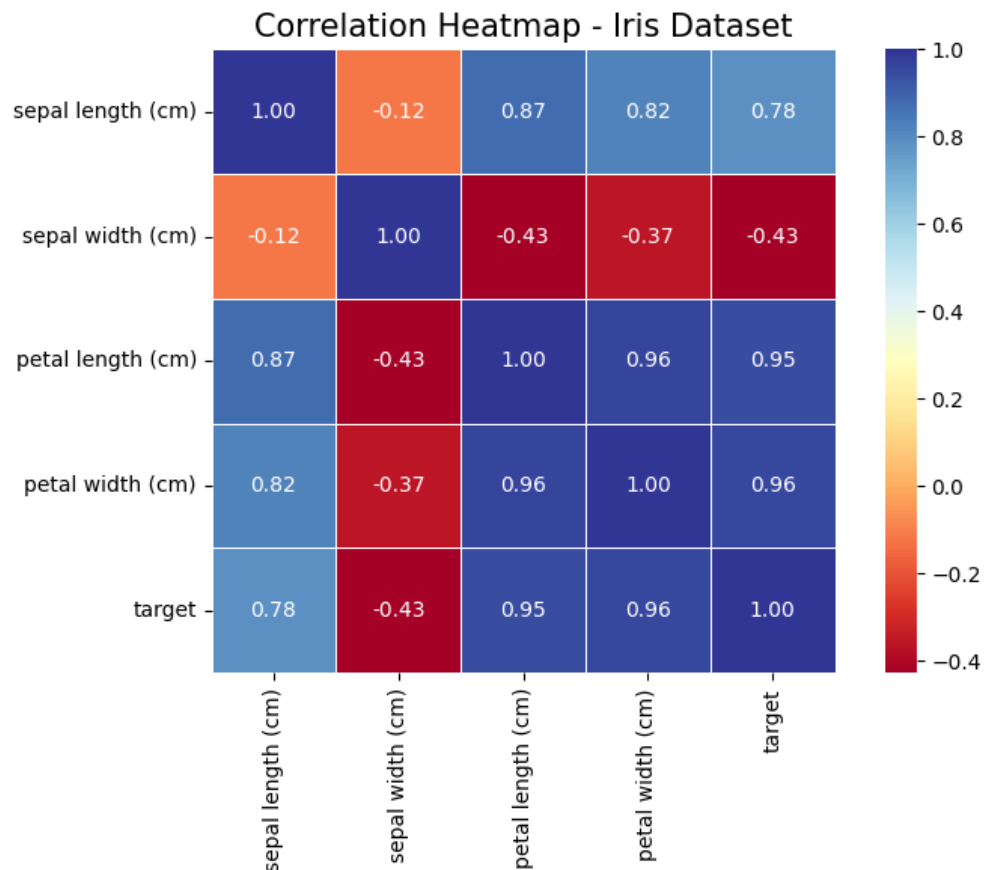


Figure 2.1: Correlation heatmap generated using Seaborn.

The heatmap makes it easy to spot that income and age move together fairly strongly in this toy dataset.

ggplot2

Brief Overview

ggplot2 is an R package by Hadley Wickham, based on Wilkinson's "Grammar of Graphics". A chart is assembled by adding layers to a base `ggplot()` call — data, aesthetic mappings, geometry, then optional themes.

Features

- Layered grammar: geoms, aesthetics, statistics and scales combined with ``+``.
- Faceting (`facet_wrap` / `facet_grid`) makes small-multiple plots trivial.

Practical Use Case

A scatter plot with a linear regression line was created — a common statistical-reporting task where ggplot2 is the tool of choice among researchers.

Advantages

- The layered grammar is elegant once it clicks and scales well to multi-facet plots.
- Publication-ready output by default — why it remains popular in academic papers.

Limitations

- Requires R, a separate learning curve for a student mainly working in Python.
- The '+' layering syntax can feel unintuitive at first.

Example Code

```
library(ggplot2)

df <- data.frame(
  study_hours = c(2, 4, 5, 7, 8, 3, 6, 9),
  score = c(50, 58, 65, 72, 80, 55, 68, 88)
)

ggplot(df, aes(x = study_hours, y = score)) +
  geom_point(color = 'darkblue', size = 2) +
  geom_smooth(method = 'lm', se = FALSE, color = 'red') +
  labs(title = 'Study Hours vs Score', x = 'Study Hours', y = 'Score') +
  theme_minimal()
```

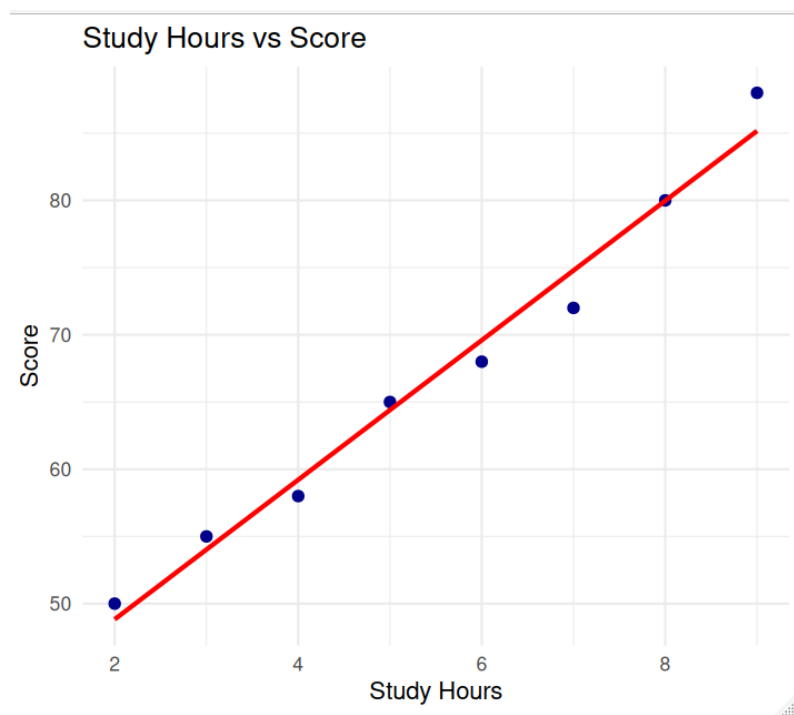


Figure 3.1: Scatter plot with regression line generated using ggplot2.

Plotly

Brief Overview

Plotly renders charts as interactive HTML/JavaScript objects instead of static images. It has bindings for Python, R and JavaScript, though `plotly.express` is the API most used in ML workflows.

Features

- Zoom, pan, and hover-tooltips work out of the box on every chart.
- Charts embed directly into web dashboards (Dash) or Jupyter notebooks.

Practical Use Case

An interactive bar chart comparing average scores across a categorical grouping was built, since hovering reveals exact values once there are many categories.

Advantages

- Interactivity is the biggest differentiator — useful for dashboards meant to be explored.
- Good default colour palettes and layout without much manual tuning.

Limitations

- Exported static images lose the interactive benefit entirely.
- Can feel heavier to render for very large datasets.

Example Code

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'branch': ['CSE', 'ICT', 'EE', 'ME'],
    'avg_score': [78, 74, 69, 66]
})

fig = px.bar(df, x='branch', y='avg_score', color='branch',
             title='Average Score by Branch')
fig.show()
```

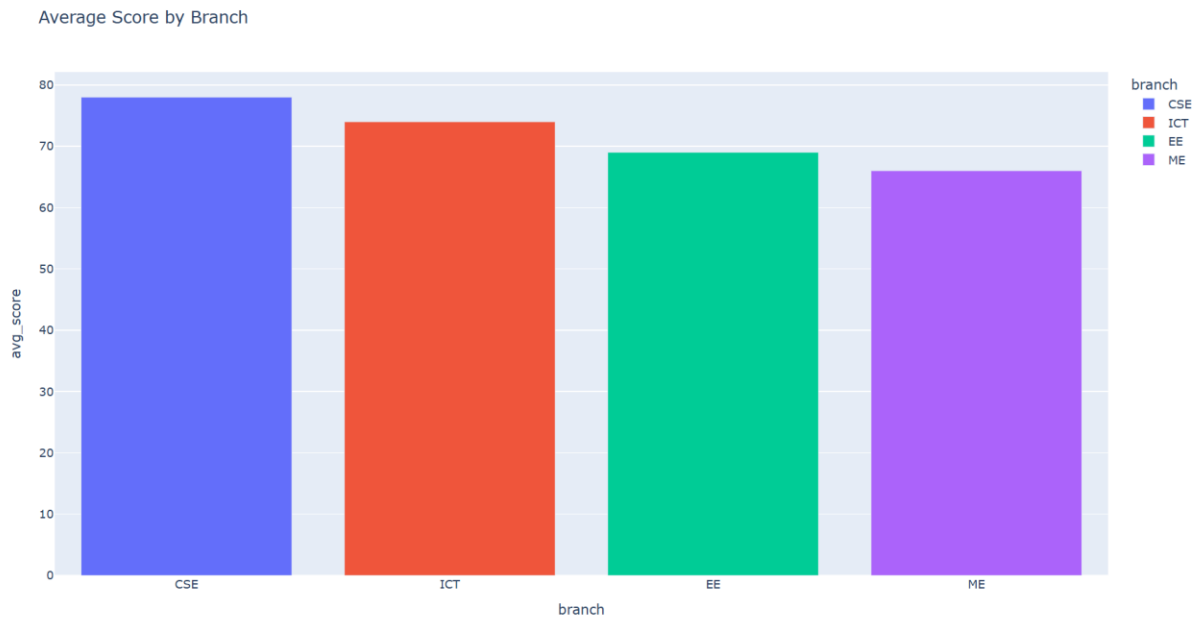


Figure 4.1: Interactive bar chart generated using Plotly.

The CSE branch shows the highest average here, and the interactive version lets a viewer hover to see the exact value.

Comparison Table

Putting all four side by side makes the trade-offs easier to see at a glance:

Tool	Language	Learning Curve	Static / Interactive	Best Use Case	Strengths	Limitations
Matplotlib	Python	Moderate	Static	Custom, low-level plots	Full control, huge flexibility	Verbose for common statistical charts
Seaborn	Python	Easy	Static	Exploratory data analysis	Great defaults, pandas integration	Limited chart variety, built on Matplotlib
ggplot2	R	Moderate–Steep	Static	Statistical / academic reporting	Elegant grammar of graphics, publication quality	Needs R, layering syntax takes getting used to
Plotly	Python / R / JS	Easy–Moderate	Interactive	Dashboards, exploratory web apps	Zoom/hover interactivity, dashboard-ready	Heavier rendering, interactivity lost in static export

Conclusion

Matplotlib gives maximum control, Seaborn speeds up exploratory analysis, ggplot2 excels at statistical reporting, and Plotly wins on interactivity. No single tool is "best" — the right pick depends on whether the job needs quick control, fast exploration, publication quality, or an interactive dashboard.

References

1. Matplotlib Documentation — <https://matplotlib.org/stable/index.html>
2. Seaborn Documentation — <https://seaborn.pydata.org/>
3. ggplot2 Documentation — <https://ggplot2.tidyverse.org/>
4. Plotly Python Documentation — <https://plotly.com/python/>